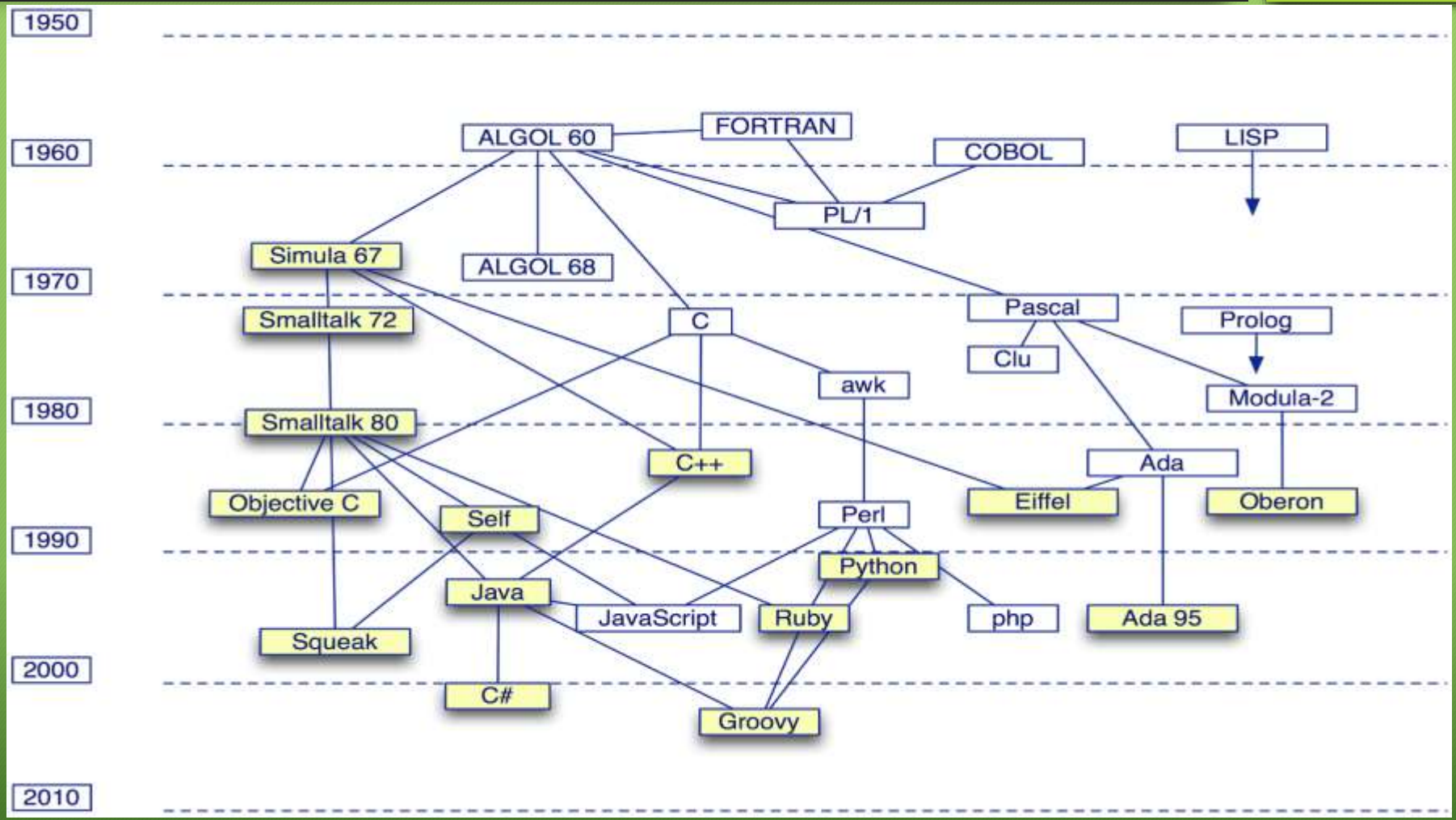
Unit 1 : Introduction

CSP215: Object Oriented Design using C++

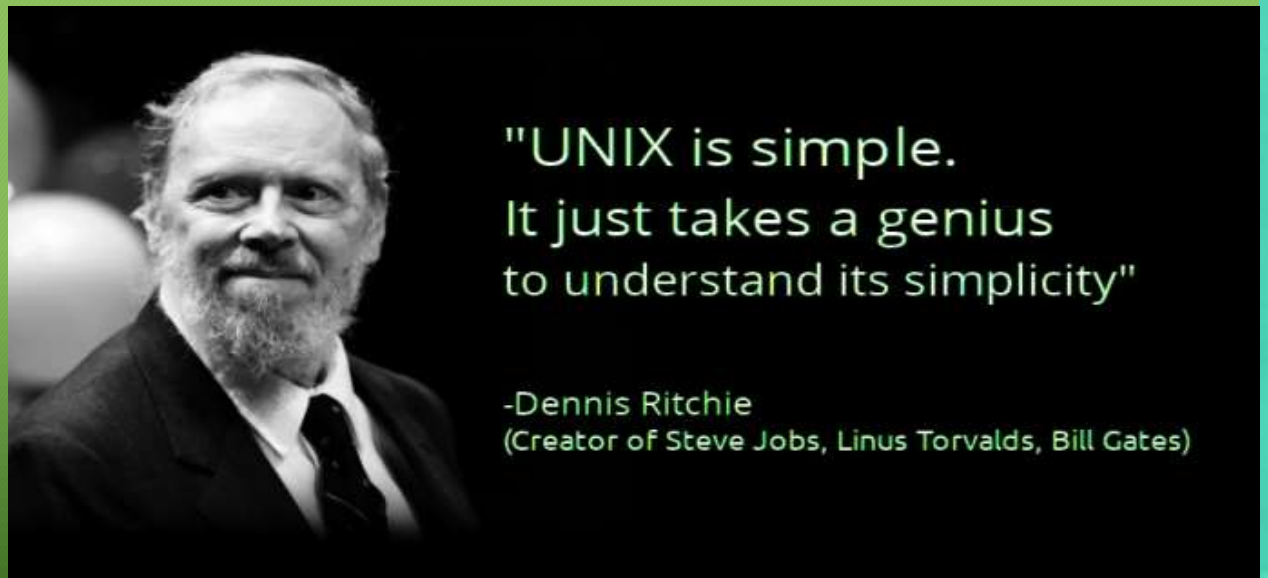
Instructor : Mr. A. S. Shelar

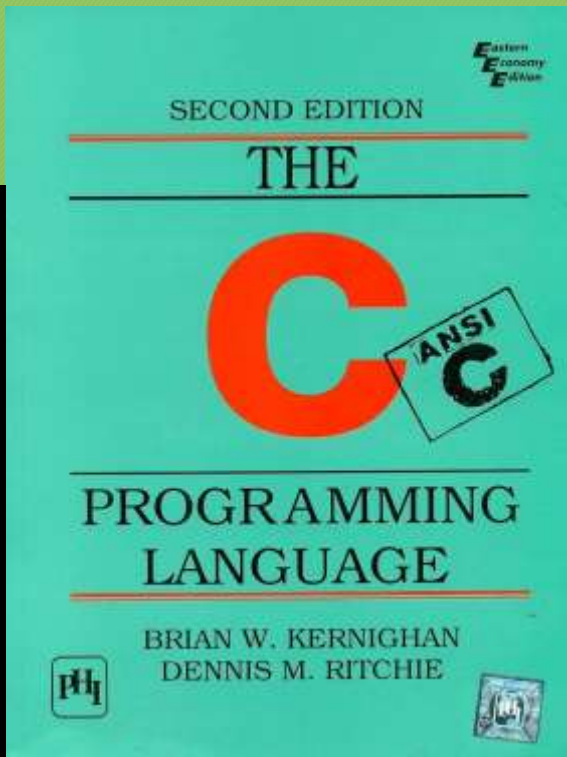
Programming language genealogy



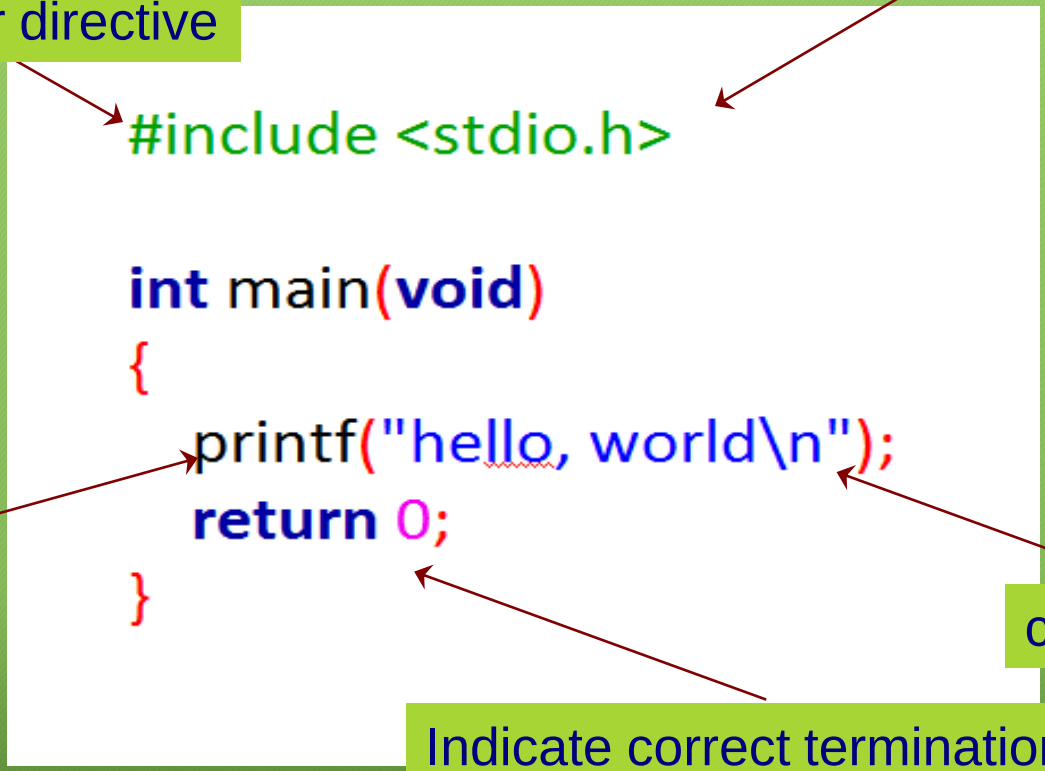
What is C ?

- C is a general purpose, procedural, imperative language developed in 1972 by Dennis Ritchie at Bell Labs for the Unix Operating System.
  - Low-level access to memory
  - Language constructs close to machine instructions
  - Used as a “machine-independent assembler”





C Program Structure



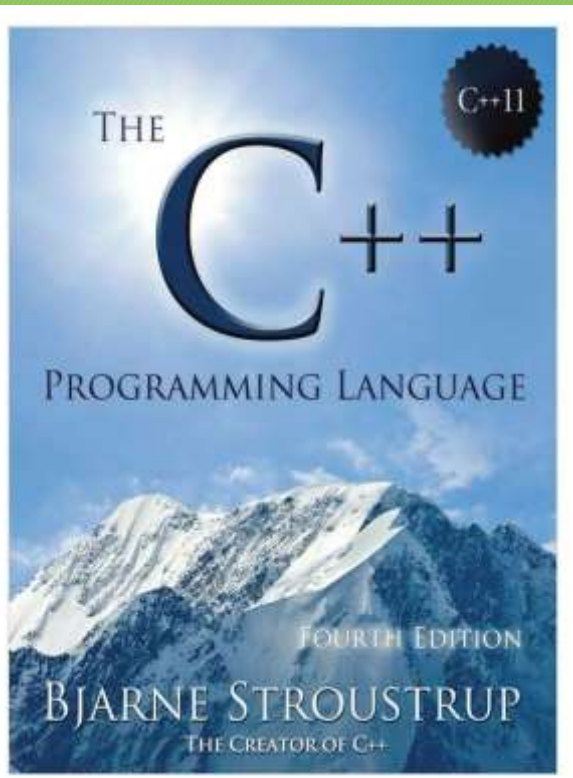
Basics of C++

What is C++ ?

- A *"better C"*

that supports:

- Systems programming
- Object-oriented programming (classes & inheritance)
- Programming-in-the-large (namespaces, exceptions)
- Generic programming (templates)
- Reuse (large class & template libraries)



C++ vs C

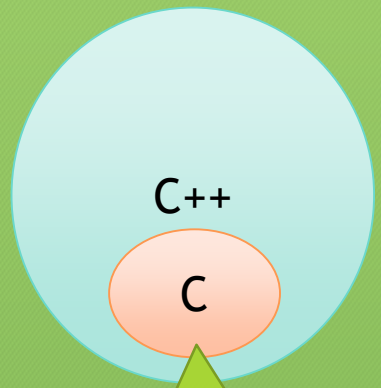
Most C programs are also C++ programs.

Nevertheless, good C++ programs usually do not resemble C:

- avoid macros (use `inline`)
- avoid pointers (use references)
- avoid `malloc` and `free` (use `new` and `delete`)
- avoid arrays and `char*` (use `vectors` and `strings`) …
- avoid `structs` (use classes)

C++ encourages a different style of programming:

- avoid procedural programming
  - *model your domain* with classes and templates



C is subset of C++ : Every program of C is by default Program of C++.

C++ Program Structure

- When you learn a computer language, you should begin by learning the basic structure for a program then only you can move on to the details, such as loops and objects.

C++ Program Structure - Example

hello.cpp

Use the standard namespace

Include standard iostream classes

A C++ comment

```
#include <iostream>
using namespace std;
// My first C++ program!
int main(void)
{
   cout << "hello world!" << endl;
   return 0;
}
```

cout is an instance of ostream

operator overloading
(two *different* argument types!)

Compilation and Execution

- Linux based System
  - g++ hello.cpp
  - ./a.out
- OR
  - g++ hello.cpp -o hello.out
  - ./hello.out

Header File Naming Convention

Kind of Header	Convention	Example	Comments
C++ old style	Ends in `.h`	`iostream.h`	Usable by C++ programs
C old style	Ends in `.h`	`math.h`	Usable by C and C++ programs
C++ new style	No extension	`iostream`	Usable by C++ programs, uses `namespace std`
Converted C	`c` prefix, no extension	`cmath`	Usable by C++ programs, might use non-C features, such as `namespace std`

Namespaces

- If you use iostream instead of iostream.h, you should use the following namespace directive to make the definitions in iostream available to your program:
  - **using namespace** std;
- This is known as using directive
- Namespace support is a fairly new C++ feature designed to simplify the writing of programs that combine preexisting code from several vendors.(to avoid collision between same class names from different vendors)
- Example:
  - dkte :: it_dept()
  - wce :: it_dept()

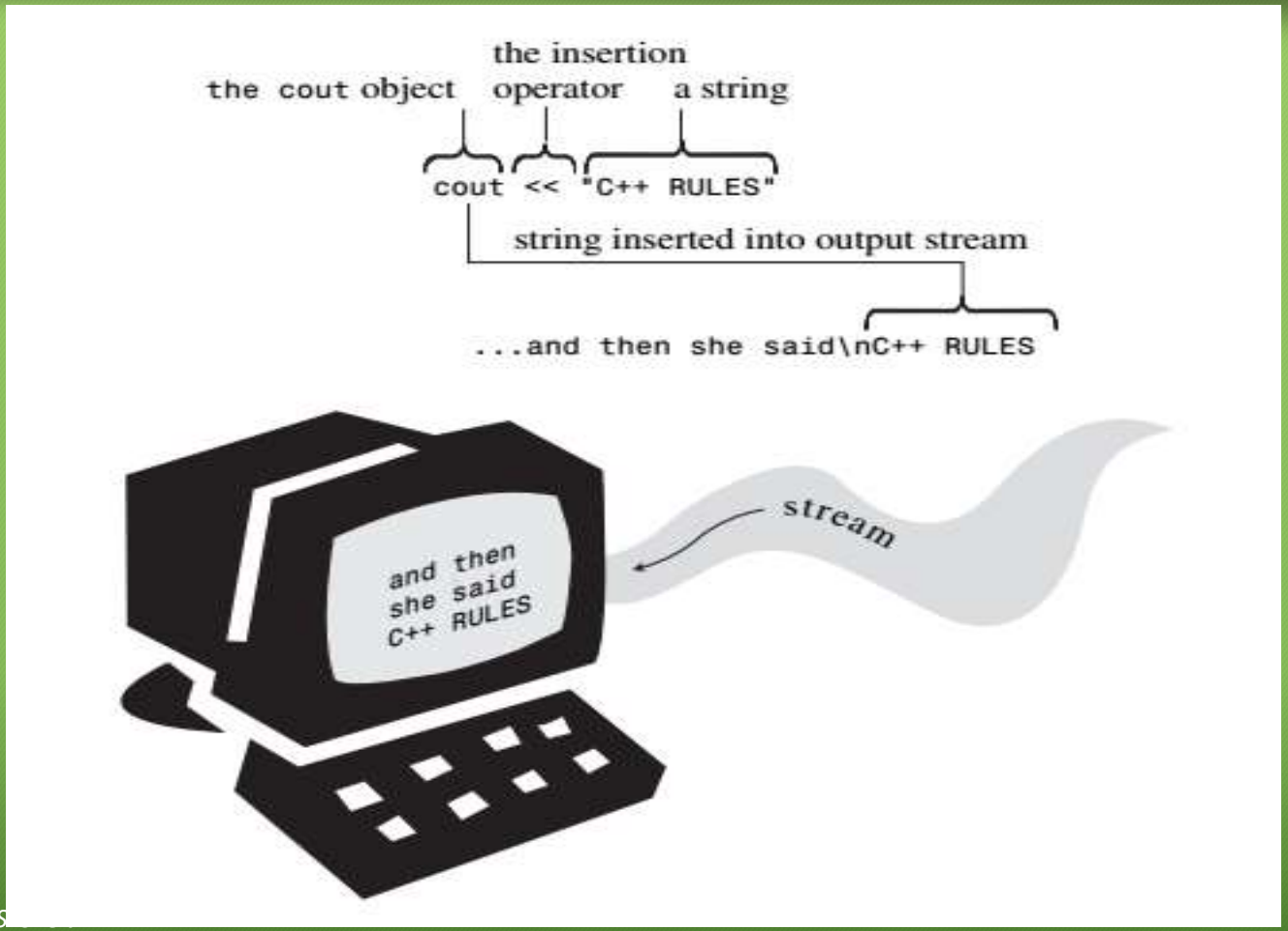
C++ Output with cout

- Now let's look at how to display a message

```
cout << "hello world!" << endl;
```

- The part enclosed within the double quotation marks is the message to print (In C++, any series of characters enclosed in double quotation marks is called a *character string*)
- The << notation indicates that the statement is sending the string to cout; the symbols point the way the information flows
- cout - It's a predefined object that knows how to display a variety of things, including strings, numbers, and individual characters.
- The cout object has a simple interface to display message.

Displaying a string by using cout.



The Manipulator endl

- cout << endl ;
- endl is a special C++ notation that represents the important concept of beginning a new line
- **Example:**

```
cout << "The Good, the";
cout << "Bad, ";
cout << "and the Ukulele";
cout << endl;
```

C++ Source Code Style

- Rules:
- One statement per line.
- An opening brace and a closing brace for a function, each of which is on its own line.
- Statements in a function indented from the braces.
- No whitespace around the parentheses associated with a function name.

Basics of C++ Program

- Programming language is a set of rules, symbols, and special words.
- Comments
  - Single Line comment //
  - Multiline comment /* …… */
- Token - The smallest individual unit of a program written in any language is called a token.
  1. **Special Symbols**
  2. **Word Symbols / Keywords**
  3. **Identifiers**

1. Special Symbols			
+	-	*	/
.	;	?	,
<=	!=	==	>=

2. Reserved Words (Word Symbols / Keywords)

and	do	mutable	template
and_eq	double	namespace	this
asm	dynamic_cast	new	throw
auto	else	not	true
bitand	enum	not_eq	try
bitor	exit()	operator	typedef
bool	explicit	or	typeid
break	export	or_eq	typename
case	extern	private	union
catch	extern "C"	protected	unsigned
char	false	public	using
class	float	register	virtual
compl	for	reinterpret_cast	void
const	friend	short	volatile
const_cast	goto	signed	void
continue	if	sizeof	wchar_t
default	inline	static	while
#define	int	static_cast	xor
delete	long	struct	xor_eq
		switch	

3. Identifiers

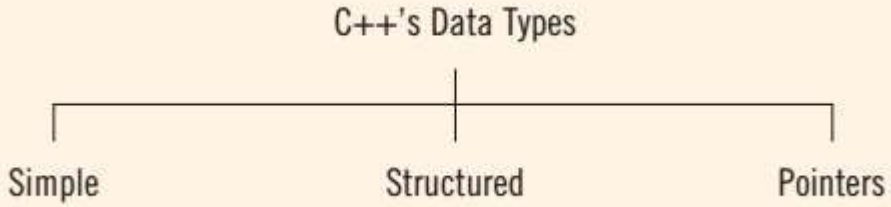
- A third category of tokens is identifiers.
- Identifiers are names of things that appear in programs, such as variables, constants, and functions.
- All identifiers must obey C++'s rules for identifiers.
- **Identifier :** A C++ identifier consists of letters, digits, and the underscore character (_) and must begin with a letter or underscore.

Example:

Illegal Identifier	Description
`employee Salary`	There can be no space between **employee** and **Salary**.
`Hello!`	The exclamation mark cannot be used in an identifier.
`one+two`	The symbol + cannot be used in an identifier.
`2nd`	An identifier cannot begin with a digit.

Data Types, Operators, Expressions

Data Types in C++



Simple Data Type:

1. Integral, which is a data type that deals with integers, or numbers without a decimal part
2. Floating-point, which is a data type that deals with decimal numbers
3. Enumeration, which is a user-defined data type

1. Integral Data Types

- Integral data types are further classified into the following nine categories: char, short, int, long, bool, unsigned char, unsigned short, unsigned int, and unsigned long.
- Newer programming languages have only five categories of simple data types: integer, real, char, bool, and the enumeration type.
- to find the maximum and minimum values/ Range of these data types we can use #include<climits> header file.

Data Type	Values	Storage (in bytes)
int	-2147483648 to 2147483647	4
bool	true and false	1
char	-128 to 127	1

The *const* Qualifier

- A symbolic name can suggest what the constant represents.
- C++ has a better way to handle symbolic constants (than #define)
- That way is to use the const keyword to modify a variable declaration and initialization
- **Example :**

```
const int MONTHS = 12;
```

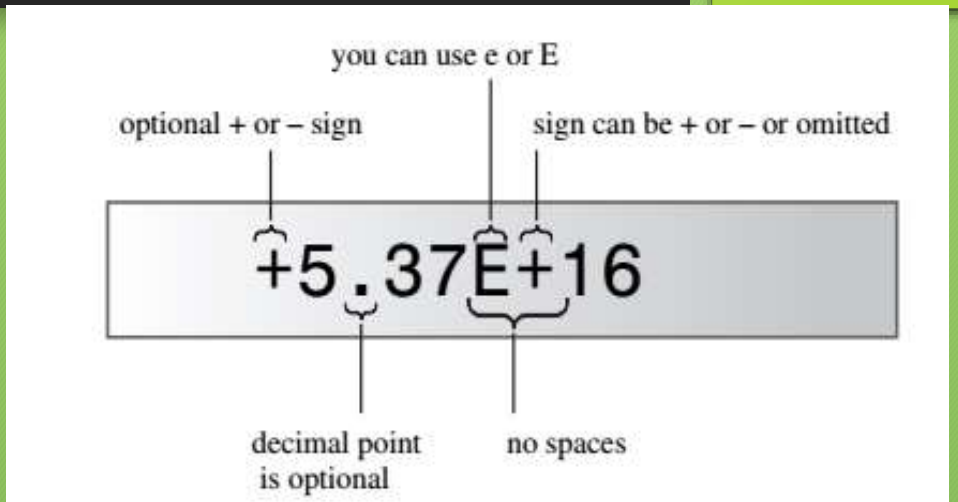
MONTHS is symbolic constant for 12

C++ Escape Sequence Codes

Character Name	ASCII Symbol	C++ Code	ASCII Decimal Code	ASCII Hex Code
Newline	NL (LF)	\n	10	0xA
Horizontal tab	HT	\t	9	0x9
Vertical tab	VT	\v	11	0xB
Backspace	BS	\b	8	0x8
Carriage return	CR	\r	13	0xD
Alert	BEL	\a	7	0x7
Backslash	\	\\	92	0x5C
Question mark	?	\?	63	0x3F
Single quote	'	\'	39	0x27
Double quote	"	\"	34	0x22

2. Floating-Point Data Types

- In the C++ floating-point notation, the letter E stands for the exponent.
- So, 16 is exponent
- 5.37 is mantissa
- **Examples:**



Real Number	C++ Floating-Point Notation
75.924	7.592400E1
0.18	1.800000E-1
0.0000453	4.530000E-5
-1.482	-1.482000E0
7800.0	7.800000E3

Data Types and Variables

- When we declare a variable, not only do we specify the name of the variable, we also specify what type of data a variable can store.
- A syntax rule to declare a variable is:
  - ✓dataType identifier;
- For example, consider the following statements:
  - int counter;
  - double interestRate;
  - char grade;

Arithmetic Operators

- Arithmetic Operator : +, -, *, / , %
- Unary operator - An operator that has only one operand.
  - Example :
    - -20
    - +30
- Binary Operator - An operator that has two operands.
  - Example :
    - 10 + 40

Order of Operation: Operator Precedence and Associativity

- When more than one arithmetic operator is used in an expression, C++ uses the operator precedence rules to evaluate the expression.
- Order of Precedence :

Precedence	Operator	Operator	Operator
High	*	/	%
low	+	-	

Operator Precedence and Associativity

- Note that the operators *, /, and % have the same level of precedence.
- Similarly, the operators + and - have the same level of precedence.
- When operators have the same level of precedence, the operations are performed from left to right associativity .
- To avoid confusion, use parentheses to group arithmetic expressions.
- **Example :**
  - 3 * 7 - 6 + 2 * 5 / 4 + 6

Expressions

There are three types of arithmetic expressions in C++:

1. Integral expressions — all operands in the expression are integers. An integral expression yields an integral result.
2. Floating-point (decimal) expressions — all operands in the expression are floating-points (decimal numbers). A floating-point expression yields a floating-point result.
3. Mixed expressions — the expression contains both integers and decimal numbers.

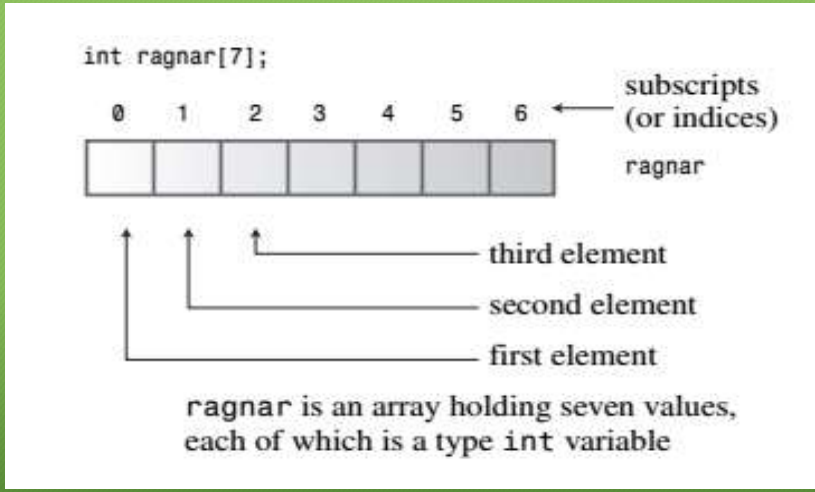
Type Conversion / Type Casting

- When a value of one data type is automatically changed to another data type, an implicit type casting is said to have occurred.
- But implicit casting may generate an unexpected values. (incorrect / loss of values).
- To avoid implicit type coercion, C++ provides for explicit type conversion through the use of a cast operator.
- **Example :**
  1. static_cast<int>(7.9) 7
  2. static_cast<int>(3.3) 3
  3. static_cast<double>(25) 25.0

Compound Data Types

Compound Types (Arrays and Strings)

- Arrays : An array is a data form that can hold several values, all of one type
- To create an array, you use a declaration statement. An array declaration should indicate three things:
  - The type of value to be stored in each element
  - The name of the array
  - The number of elements in the array



`ragnar` is an array holding seven values, each of which is a type `int` variable

Arrays ….

- You can use the initialization form only when defining the array. You cannot use it later
- you cannot assign one array wholesale to another
- **Example:**

```
int cards[4] = {3, 6, 8, 10};        // okay
int hand[4];                         // okay
hand[4] = {5, 6, 7, 9};              // not allowed
hand = cards;                        // not allowed
```

Arrays ….

- When initializing an array, you can provide fewer values than array elements.
- E.g.

```
float hotelTips[5] = {5.0, 2.5};
```

- If you partially initialize an array, the compiler sets the remaining elements to zero
- Thus, it's easy to initialize all the elements of an array to zero—just explicitly initialize the first element to zero and then let the compiler initialize the remaining elements to zero.
- E.g.

```
long totals[500] = {0};
```

Arrays ….

- Question:

1. long totals[100] = {1};

- Answer : if you initialize to {1} instead of to {0}, just the first element is set to 1; the rest still get set to 0.

Strings

- A string is a series of characters stored in consecutive bytes of memory
- C++ has two ways of dealing with strings.
  1. C-style string
  2. String class.
- **Example:**

```
char dog [5] = { 'b', 'e', 'a', 'u', 'x'};      // not a string!
char cat[5] = {'f', 'a', 't', 's', '\0'};       // a string!
```

Strings

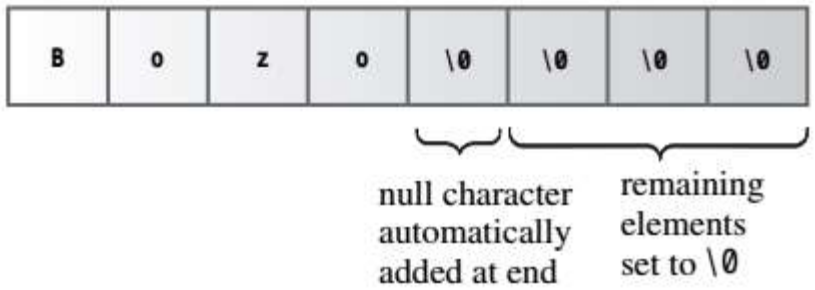
- There is a better way to initialize a character array to a string. Just use a **quoted string**, called a string constant or string literal.
- **Example:**

```
char bird[10] = "Mr. Cheeps";      // the \0 is understood
char fish[] = "Bubbles";           // let the compiler count
```

- Quoted strings always include the terminating null character **implicitly.**
- **Question :**

1. **What is length of boss?**

```
char boss[8] = "Bozo";
```



```cpp
// strings.cpp -- storing strings in an array
#include <iostream>
#include <cstring> // for the strlen() function
int main()
{
    using namespace std;
    const int Size = 15;
    char name1[Size]; // empty array
    char name2[Size] = "C++owboy"; // initialized array
    // NOTE: some implementations may require the static keyword
    // to initialize the array name2
    cout << "Howdy! I'm " << name2;
    cout << "! What's your name?\n";
    cin >> name1;
    cout << "Well, " << name1 << ", your name has ";
    cout << strlen(name1) << " letters and is stored\n";
    cout << "in an array of " << sizeof(name1) << " bytes.\n";
    cout << "Your initial is " << name1[0] << ".\n";
    name2[3] = '\0'; // null character
    cout << "Here are the first 3 characters of my name: ";
    cout << name2 << endl;
    return 0;
}
```

Babooooo

String..

- The cin technique is to use whitespace—spaces, tabs, and newlines—to delineate a string. This means cin reads just one word when it gets input for a character array.
- After it reads this word, cin automatically adds the terminating null character when it places the string into the array.

Reading String Input a Line at a Time

- for the istream class, of which cin is an example, has some line-oriented class member functions: getline() and get().
- Both read an entire input line— that is, up until a newline character. However, **getline() then discards the newline character, whereas get() leaves it in the input queue.**
- **Example:**

```
1  // read_string.cpp -- reading more than one word with getline
2  #include <iostream>
3  int main()
4  {
5  using namespace std;
6  const int ArSize = 20;
7  char name[ArSize];
8  char dessert[ArSize];
9  cout << "Enter your name:\n";
10 cin.getline(name, ArSize); // reads through newline
11 cout << "Enter your favorite dessert:\n";
12 cin.getline(dessert, ArSize);
13 cout << "I have some delicious " << dessert;
14 cout << " for you, " << name << ".\n";
15 return 0;
16 }
```

- Question :
- How to read two reads two consecutive input lines into the arrays ?
- Answer :

```
cin.getline(name1, ArSize).getline(name2, ArSize);
```

String Class

Introducing the *string* Class

- The ISO/ANSI C++ Standard expanded the C++ library by adding a string class.
- So now, instead of using a character array to hold a string, you can use a type string variable
- use #include<string> to use string as class in C++;

```cpp
// strtype1.cpp -- using the C++ string class
#include <iostream>
#include <string> // make string class available
int main()
{
    using namespace std;
    char charr1[20]; // create an empty array
    char charr2[20] = "jaguar"; // create an initialized array
    string str1; // create an empty string object
    string str2 = "panther"; // create an initialized string
    cout << "Enter a kind of feline: ";
    cin >> charr1;
    cout << "Enter another kind of feline: ";
    cin >> str1; // use cin for input
    cout << "Here are some felines:\n";
    cout << charr1 << " " << charr2 << " "
    << str1 << " " << str2 // use cout for output
    << endl;
    cout << "The third letter in " << charr2 << " is "
    << charr2[2] << endl;
    cout << "The third letter in " << str2 << " is "
    << str2[2] << endl; // use array notation
    return 0;
}
```

Assignment, Concatenation :


```
// strtype2.cpp -- assigning, adding, and appending
#include <iostream>
#include <string> // make string class available
int main()
{
using namespace std;
string s1 = “penguin”;
string s2, s3;

cout << “You can assign one string object to another: s2 = s1\n”;

s2 = s1;
cout << “s1: “ << s1 << “, s2: “ << s2 << endl;
cout << “You can assign a C-style string to a string object.\n”;
cout << “s2 = \”buzzard\”\n”;

s2 = “buzzard”;
cout << “s2: “ << s2 << endl;
cout << “You can concatenate strings: s3 = s1 + s2\n”;

s3 = s1 + s2;
cout << “s3: “ << s3 << endl;
cout << “You can append strings.\n”;

s1 += s2;
cout <<”s1 += s2 yields s1 = “ << s1 << endl;

s2 += “ for a day”;
cout <<”s2 += \” for a day\” yields s2 = “ << s2 << endl;

return 0;
}
```

Read Line using string class:

1. Read single line :

```
string name2;
cout<<"Enter some text"<<endl;
getline(cin,name2);
```

2. Continuous no of lines

Input :

```
String line;
while (getline(cin, line))
{
    if (line == "*")
    break;

    line += " " + line;
```

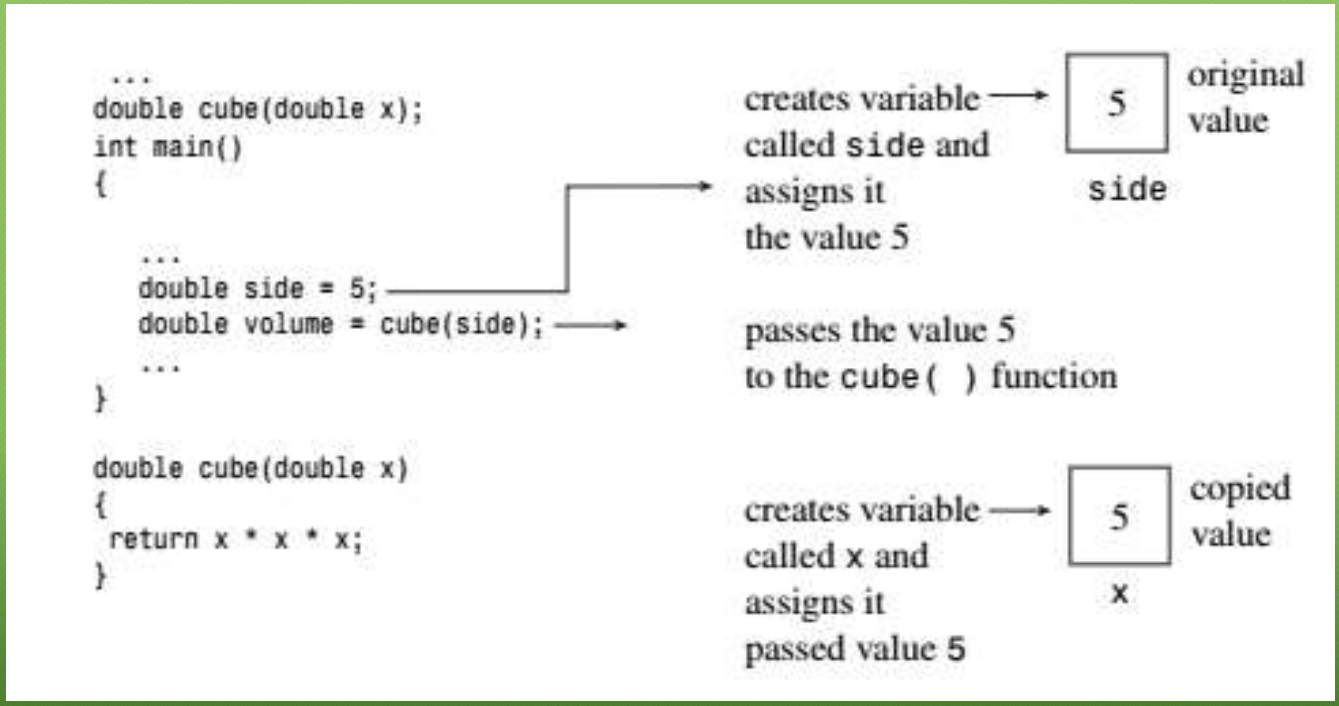
Basics of Function in C++

Function in C++

- A function is a group or block of statements that together perform a specific task
  - User defined function
  - Inbuilt function (standard library function)

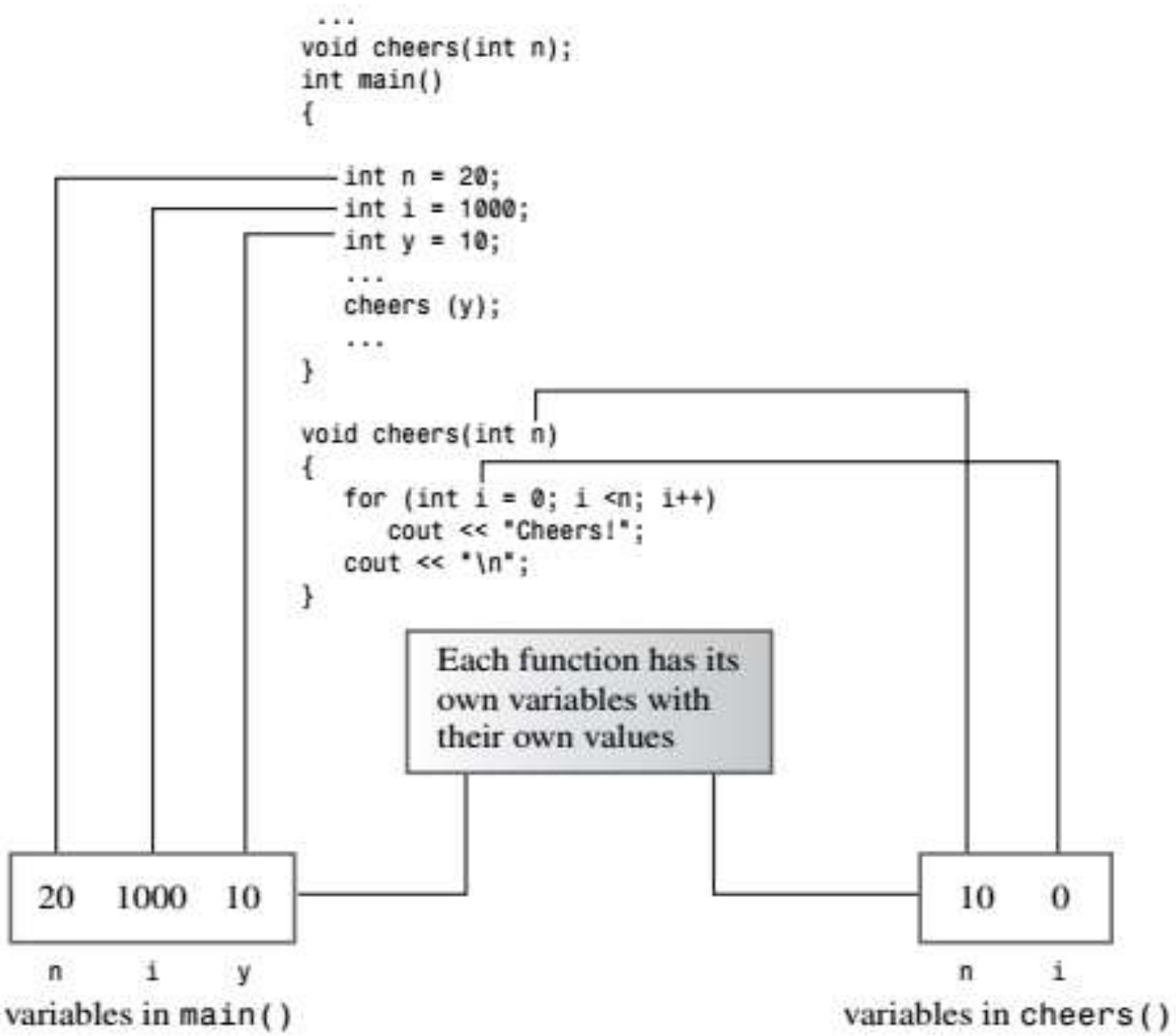
Function Arguments and Passing by Value

- C++ normally passes arguments by value.
- That means the numeric value of the argument is passed to the function, where it is assigned to a new variable



Actual and Formal parameters

- A variable that's used to receive passed values is called a *formal argument* or *formal parameter*
- The value passed to the function is called the *actual argument* or *actual parameter*.
- To simplify, the C++ Standard uses the word *argument* by itself to denote an actual argument or parameter and the word *parameter* by itself to denote a formal argument or parameter
- Local Variables or automatic variables


...
void cheers(int n);
int main()
{
int n = 20;
int i = 1000;
int y = 10;
...
cheers (y);
...
}
void cheers(int n)
{
for (int i = 0; i <n; i++)
cout << "Cheers!";
cout << "\n";
}
Each function has its own variables with their own values
20 1000 10
n i y
variables in main()
10 0
n i
variables in cheers()

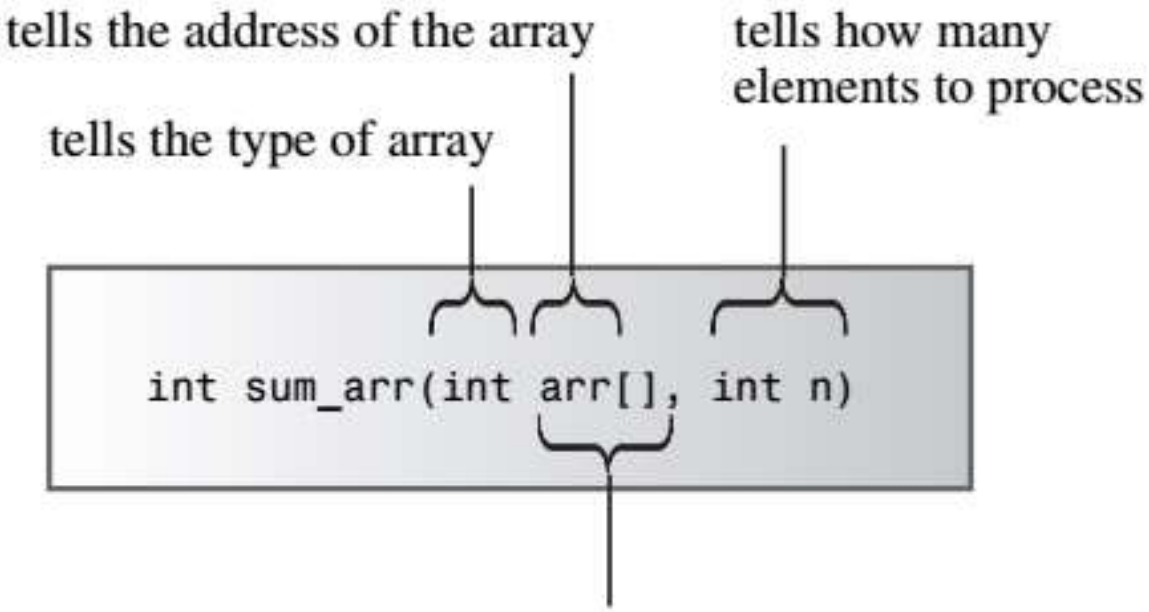
Functions and Arrays

- Passing an array as argument.

```
int sum_arr(int arr[], int n) // arr = array name, n = size
```

- The brackets seem to indicate that arr is an array, and the fact that the brackets are empty seems to indicate that you can use the function with an array of any size.
- But things are not always as they seem: arr is not really an array; it's a pointer!

The Implications of Using Arrays as Arguments



Functions and Two-Dimensional Arrays

Example:

- Function call

```
int data[3][4] = {{1,2,3,4}, {9,8,7,6}, {2,4,6,8}};
int total = sum(data, 3);
```

- Prototype

```
int sum(int (*ar2)[4], int size);
```

OR

```
int sum(int ar2[][4], int size);
```

Structure in C++

- The struct keyword defines a structure type and/or a variable of a structure type.
- A structure type is a user-defined composite type.
- It is composed of fields or members that can have different types.
- In C++, a structure is the same as a class except that its members are public by default.

```cpp
#include <iostream>
using namespace std;

struct PERSON {    // Declare PERSON struct type
    int age;    // Declare member types
    long ss;
    float weight;
    char name[25];
} family_member;    // Define object of type PERSON


int main() {
    struct PERSON sister;    // C style structure declaration
    PERSON brother;    // C++ style structure declaration
    sister.age = 13;    // assign values to members
    brother.age = 7;
    cout << "sister.age = " << sister.age << '\n';
    cout << "brother.age = " << brother.age << '\n';

}
```

Functions and Structures

- **Passing and Returning Structures**
- Question :
- Some maps will tell you that it is 4 hours, 50 minutes, from Ichalkaranji to Pune City and 1 hour, 25 minutes, from Pune to Shirur. You can use a structure to represent such times, using one member for the hour value and a second member for the minute value.
- Now, assume day1 is ichalkarnji to pune and day2 is pune to shirur . So, calculate total travel time of your Journey.

Adventures in Function

Adventures in Functions

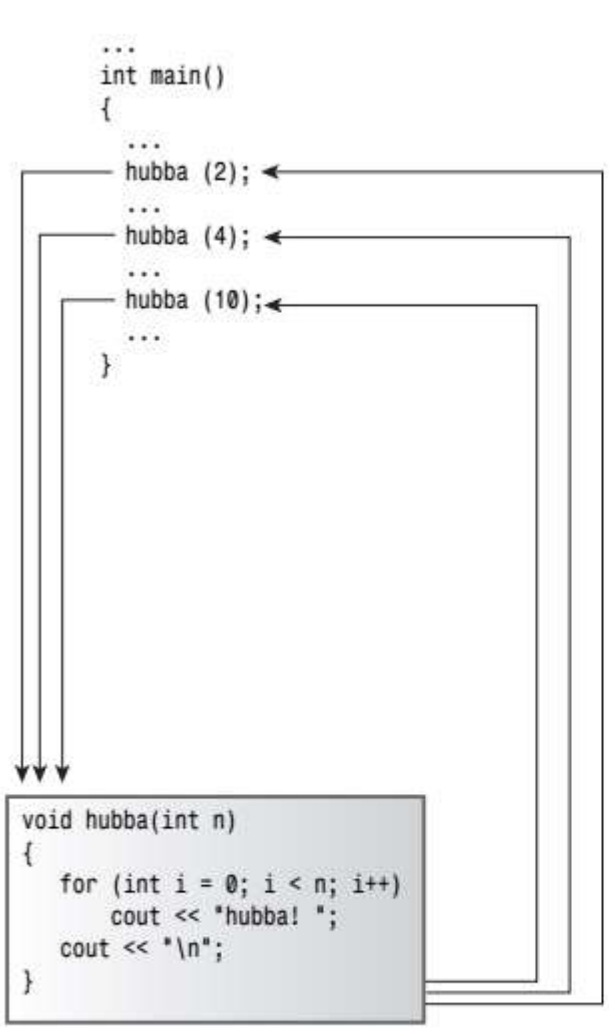
- C++ Inline Functions :**Inline functions** are a C++ enhancement designed to speed up programs
- The primary distinction between normal functions and inline functions is not in how you code them but in how the C++ compiler incorporates them into a program.
- The final product of the compilation process is an executable program, which consists of a set of machine language instructions.
- When you start a program, the operating system loads these instructions into the computer's memory so that each instruction has a particular memory address.
- The computer then goes through these instructions step-by-step.

- When a program reaches the function call instruction, the program stores the memory address of the instruction immediately following the function call, copies function arguments to the stack (a block of memory reserved for that purpose), jumps to the memory location that marks the beginning of the function, executes the function code (perhaps placing a return value in a register), and then jumps back to the instruction whose address it saved. Jumping back and forth and keeping track of where to jump means that there is an overhead in elapsed time to using functions.

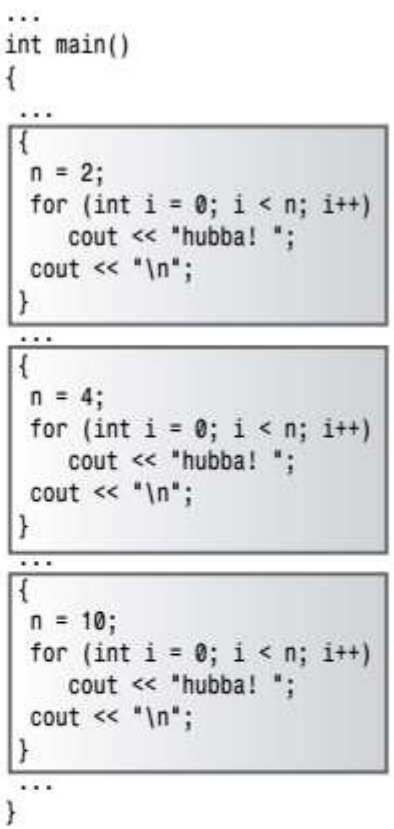
C++ Inline Function:

- **C++ inline functions** provide an alternative. In an inline function, the compiled code is "inline" with the other code in the program
- That is, the compiler replaces the function call with the corresponding function code.
- With inline code, the program doesn't have to jump to another location to execute the code and then jump back.
- Inline functions thus run a little faster than regular functions, but they come with a memory penalty.
- If a program calls an inline function at 10 separate locations, then the program winds up with 10 copies of the function inserted into the code.

Inline functions versus regular functions.



A regular function transfers program execution to a separate function.



An inline function replaces a function call with inline code.

- To use this feature, you must take at least one of two actions:
  1. Preface the function declaration with the keyword inline.
  2. Preface the function definition with the keyword inline

```
// inline.cpp -- using an inline function
#include <iostream>
// an inline function definition
inline double square(double x) { return x * x; }
int main()
{
    using namespace std;
    double a, b;
    double c = 13.0;
    a = square(5.0);
    b = square(4.5 + 7.5); // can pass expressions
    cout << "a = " << a << ", b = " << b << "\n";
    cout << "c = " << c;
    cout << ", c squared = " << square(c++) << "\n";
    cout << "Now c = " << c << "\n";
    return 0;
}
```

References as Function Parameters

- References are used as function parameters, making a variable name in a function an alias for a variable in the calling program.
- This method of passing arguments is called passing by reference.
- Passing by reference allows a called function to access variables in the calling function.

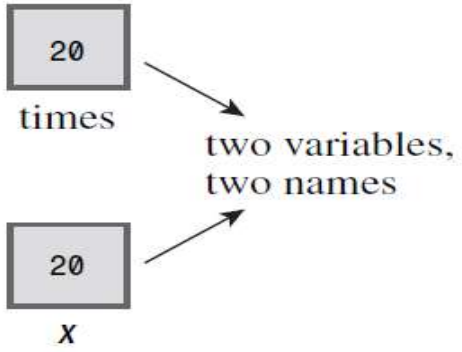
Passing by value

```
void sneezy(int x);
int main()
{
    int times = 20;
    sneezy(times);
    ...
}

void sneezy(int x)
{
   ...
}
```

creates a variable called times, assigns it the value of 20

creates a variable called x, assigns it the passed value of 20



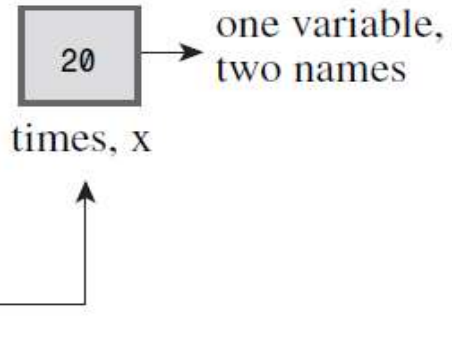
Passing by reference

```
void grumpy(int &x);
int main()
{
    int times = 20;
    grumpy(times);
    ...
}

void grumpy(int &x)
{
   ...
}
```

creates a variable called times, assigns it the value of 20

makes x an alias for times



Default Argument of function

- A *default argument* is a value that's used automatically if you omit the corresponding actual argument from a function call.
- **Example 1:** if you set up void wow(int n=1) so that n has a default value of 1,the function call wow() is the same as wow(1).
- **Example 2:** the harpo() prototype permits calls with one, two, or three arguments:

```
int harpo(int n, int m = 4, int j = 5);          // VALID
int chico(int n, int m = 6, int j);              // INVALID
int groucho(int k = 1, int m = 2, int n = 3);    // VALID
```

- This gives you flexibility in how you use a function.

```
beeps = harpo(2);        // same as harpo(2,4,5)
beeps = harpo(1,8);      // same as harpo(1,8,5)
beeps = harpo (8,7,6);   // no default arguments used

beeps = harpo(3, ,8);    // invalid, doesn't set m to 4
```

The actual arguments are assigned to the corresponding formal arguments from left to right; you can't skip over arguments

Function Overloading

- *function overloading* also called *function polymorphism*.
- Use multiple functions sharing the same name.
- The word *polymorphism* means having many forms, so *function polymorphism* lets a function have many forms.
- Similarly, the expression *function overloading* means you can **attach more than one function to the same name**, thus overloading the name.
- The key to function overloading is a function's argument list, also called the function signature.

Function Overloading

- **Example : A set of print() function with following prototypes.**

```
void print(const char * str, int width);   // #1
void print(double d, int width);           // #2
void print(long l, int width);             // #3
void print(int i, int width);              // #4
void print(const char *str);               // #5
```

- **Function Call**

```
print("Pancakes", 15);          // use #1
print("Syrup");                 // use #5
print(1999.0, 10);              // use #2
print(1999, 12);                // use #4
print(1999L, 15);               // use #3
```

Function Templates

- A *function template* is a generic function description.
- It defines a function in terms of a generic type for which a specific type, such as int , double or any type, can be substituted.
- By passing a type as a parameter to a template, you cause the compiler to generate a function for that particular type.
- It is sometimes termed generic programming

```cpp
#include <iostream>
using namespace std;

template<typename Any>
Any add(Any ,Any );//function prototype

int main()
{
    int number1=100, number2=200;
    int result1=add(number1,number2); //function call using two int argume
    cout<<"Addition of two integers numbers : "<<result1<<endl;

    double d1=100.50, d2=199.50;
    double result2=add(d1,d2); //function call using two real arguments
    cout<<"Addition of two floating numbers :"<<result2<<endl;

    return 0;
}
template<typename Any>
Any add(Any  no1,Any  no2) //function definition
{
    return no1+no2;
}
```

1. Function call : add Two integers numbers

2. Function call : add Two real numbers

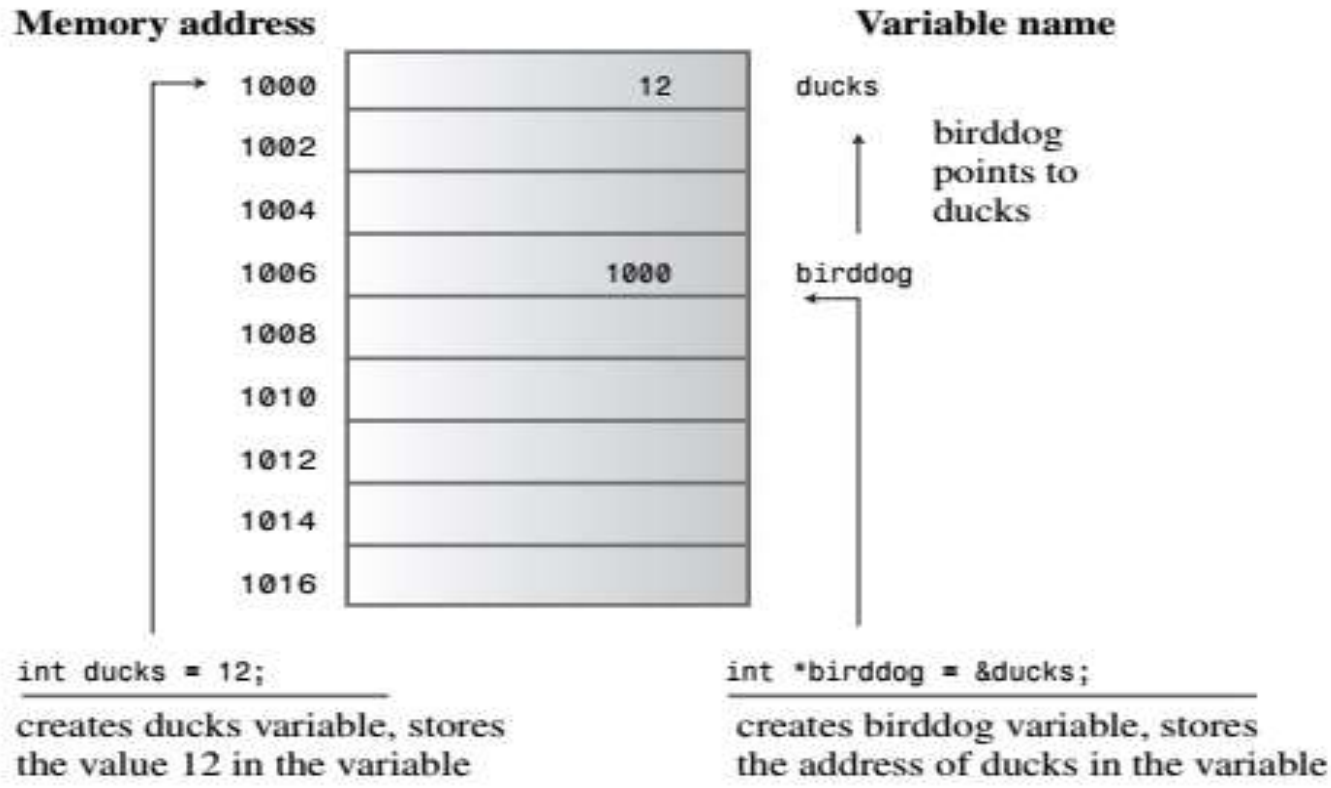
Function Template : Same for both calls

Dynamic Memory Allocation

Pointers and the Free Store

- Three fundamental properties of which a computer program must keep track when it stores data.
  - Where the information is stored
  - What value is kept there
  - What kind of information is stored
- You've used one strategy for accomplishing these ends:
  - By defining a simple variable. The declaration statement provides the type and a symbolic name for the value.
  - It also causes the program to allocate memory for the value and to keep track of the location internally

Pointers store addresses.



- **Pointer Danger**

```
long * fellow;          // create a pointer-to-long
*fellow = 223323;       // place a value in never-never land
```

- Pointer Golden Rule: Always initialize a pointer to a definite and appropriate address before you apply the dereferencing operator (*) to it.

Allocating Memory with new

- let's see how pointer can implement that important OOP technique of allocating memory as a program runs.
- So far, you've initialized pointers to the addresses of variables; the variables are named memory allocated during compile time, and each pointers merely provides an alias for memory you could access directly by name anyway.
- The true worth of pointers comes into play when you allocate unnamed memory during runtime to hold values.
- The key is the C++ new operator. You tell new for what data type you want memory; new finds a block of the correct size and returns the address of the block.
- Example :

```
int * pn = new int;
```

```cpp
// use_new.cpp -- using the new operator
#include <iostream>
int main()
{
        using namespace std;
        int * pt = new int; // allocate space for an int
        *pt = 1001; // store a value there
        cout << "int ";
        cout << "value = " << *pt << ": location = " << pt << endl;
        double * pd = new double; // allocate space for a double
        *pd = 10000001.0; // store a double there
        cout << "double ";
        cout << "value = " << *pd << ": location = " << pd << endl;
        cout << "size of pt = " << sizeof(pt);
        cout << ": size of *pt = " << sizeof(*pt) << endl;
        cout << "size of pd = " << sizeof pd;
        cout << ": size of *pd = " << sizeof(*pd) << endl;
        return 0;
}
```

Freeing Memory with delete

- The delete operator, which enables you to return memory to the memory pool when you are finished with it.
- That is an important step toward making the most effective use of memory.
- Memory that you return, or free, can then be reused by other parts of the program.
- **Example:**

```
int * ps = new int; // allocate memory with new

/* use the memory
....code
....code
....code          */

delete ps; // free memory with delete when done
```

Memory Leakage

- delete removes the memory to which ps points; it doesn't remove the pointer ps itself.
- You can reuse ps, for example, to point to another new allocation.
- You should always balance a use of new with a use of delete; otherwise, you can wind up with a memory leak.
- Memory leak means, memory that has been allocated but can no longer be used.
- If a memory leak grows too large, it can bring a program seeking more memory to a halt.

- You should not attempt to free a block of memory that you have previously freed.
- The C++ Standard says the result of such an attempt is undefined.
- Also, you cannot use delete to free memory created by declaring ordinary variables:

```
int * ps = new int; // ok
delete ps; // ok
delete ps; // not ok now
int jugs = 5; // ok
int * pi = &jugs; // ok
delete pi; // not allowed, memory not allocated by new
```

Using new to Create Dynamic Arrays

- Allocating the array during compile time is called static binding, meaning that the array is built in to the program at compile time.
- But with new, you can create an array during runtime if you need it and skip creating the array if you don't need it. Or you can select an array size after the program is running.
- This is called dynamic binding, meaning that the array is created while the program is running. Such an array is called a dynamic array.
- With dynamic binding, the program can decide on an array size while the program runs.

```cpp
// arraynew.cpp -- using the new operator for arrays
#include <iostream>
using namespace std;
int main()
{
        double * p3 = new double [3]; // space for 3 doubles

        p3[0] = 0.2; // treat p3 like an array name
        p3[1] = 0.5;
        p3[2] = 0.8;
        cout << "p3[1] is " << p3[1] << ".\n";
        p3 = p3 + 1; // increment the pointer
        cout << "Now p3[0] is " << p3[0] << " and ";
        cout << "p3[1] is " << p3[1] << ".\n";
        p3 = p3 - 1; // point back to beginning

        delete [] p3; // free the memory

        return 0;
}
```

Summary :

- Now, You know about basic data types in C++ and how to store a data inside these datatypes as well as how to use these data types.
- Function is the main turning point of the programming which tells you what is function , what is use of it and importantly how to write a function.
- Function again gives you some adventures features like reference parameter, inline function, default arguments to function, how to pass array, structure to function, scope of the variables etc.
- Dynamic Memory allocation which gives you how to deal with memory at runtime and use that memory, expand the memory and deallocate it explicitly in C++.